

NAVIGATION AND THE COMMAND CENTRE, REBUILT.

A comprehensive redesign of the public navigation and the Command Centre, on the web and in the iOS app. The bar is gorgeous, architecturally and technically impressive — the kind of work a lifelong front-end developer would look at and want to read the source of.

This document is written for the engineer or agent who will execute it. It is opinionated on purpose: it names the diagnosis, the direction, the architecture, the sequence, and the proof required at each stage. Where it says **must**, the project already has a test that will fail if you ignore it.

0. HOW TO READ THIS

SECTION	WHAT IT GIVES YOU
1. Diagnosis	What is actually wrong, with counts from the codebase
2. Direction	The aesthetic and the principles that settle arguments
3. Architecture	The token pipeline — the spine of the whole job
4–8. Workstreams	A–E, each with files, deliverables and proof
9. Sequence	The order to do it in, and what can run in parallel
10. Working notes	Verification recipes, guardrails, environment facts

Do not start at section 4. Sections 2 and 3 are what make the rest coherent; skipping them produces another layer of one-off styling on top of the two that are already there.

1. DIAGNOSIS

1.1 The iOS Command Centre is a stock iOS settings app with a dark tint

`ios/XertFitnessApp/XertFitnessApp/Views/AdminCommandCentreView.swift` is **18,009 lines**. Counted constructs:

CONSTRUCT	COUNT	WHAT IT MEANS
<code>Section(</code>	130	Stock grouped-list sections
<code>List { / List(</code>	35	Stock table views
<code>Form {</code>	13	Stock settings forms
<code>.listRowBackground</code>	174	Repainting stock rows one at a time
<code>.navigationTitle</code>	51	Stock nav bars
<code>.buttonStyle(.bordered...</code>	78	Stock buttons
<code>Picker(</code>	56	Stock wheels and menus
<code>TextField(</code>	49	Unstyled fields
<code>ScrollView</code>	4	Custom layout
<code>LazyVStack</code>	1	Custom layout

That ratio *is* the complaint. Nothing here is broken — it is a competent stock-SwiftUI app — but 174 `.listRowBackground` calls are the sound of someone repainting `UITableView` one cell at a time. It reads as an old iOS app because structurally it **is** one.

The cruellest detail: a design system already exists at

`ios/XertFitnessApp/XertFitnessApp/AdminDesignSystem.swift` (117 lines — `XertSpace`, `xertOwnerScreen()`, `xertOwnerCard()`, `XertOwnerHeading`, `XertOwnerRow`, `XertOwnerEmptyState`).

Adoption inside the Command Centre:

- `xertOwnerScreen()` — **40** uses (this only paints a background *behind* the stock list)
- `xertOwnerCard()` — **0**
- `xertOwnerContentPadding()` — **0**
- `XertOwnerHeading` / `XertOwnerRow` / `XertOwnerEmptyState` — **1 each**

So the system is applied as a paint colour and never as structure. Fixing this is the single highest-leverage change in the entire brief.

By contrast the **member** side of the app has already been modernised — `RootView.swift` hides the system tab bar (`.toolbar(.hidden, for: .tabBar)`) and renders a custom `navigationDock` and `navigationRail`. That is the standard the owner side has to reach.

1.2 The web is further along, and the gap is polish and hierarchy, not structure

`src/components/admin/ui.jsx` is a real kit (`ADMIN_PAGE` , `ADMIN_TEXT` , `ADMIN_INPUT` , `ADMIN_BUTTON` , `ADMIN_PANEL` , `AdminPageHeader`), and `test/admin-ui-kit.test.js` actively prevents hand-rolled buttons, inputs, titles and raw hex from creeping back. `AdminLayout.jsx` (375 lines) already has hubs, a sidebar, a workspace tab rail, a mobile dock and a command palette across 24 sections (`src/lib/adminNavigation.js`).

What is actually wrong, from screenshots taken at 390×844 and 1440×900:

Public navbar — desktop. A 56px bar (`h-14`) on a 1440px viewport, with the logo pinned hard to the top edge. Seven nav links, "Log In" and a "Book Now" pill all sit in one flat row with no grouping, so a navigation link and an account action are given identical visual weight. The active state is colour-only — steel text, no indicator — which at `text-sm` on a dark ground is close to invisible. The primary pill carries a heavy diffuse glow that dates it.

Public navbar — mobile. Six items in Bebas Neue at 2.375rem, each with a chevron. The chevrons promise a submenu and every one of them is a leaf link — a false affordance. Below CONTACT there is roughly **400px of dead space** before the pinned Log In / Book Now panel. There is no account state, no next class, no phone number, nothing. On a gym site opened one-handed at 5am, that empty half is the most valuable space on the screen.

1.3 Root cause

Two design systems exist (`ui.jsx` , `AdminDesignSystem.swift`), they do not know about each other, and neither is derived from a shared source. `tailwind.config.js` hard-codes the palette; `Theme.swift` hard-codes it again. Nothing keeps them in sync, so "the XERT blue" is a value that has to be remembered rather than imported. Every future divergence is free.

2. DIRECTION

2.1 The aesthetic: industrial precision

XERT is a strength and conditioning facility, not a boutique wellness studio. The visual language should feel like **well-made equipment**: confident, high-contrast, precisely aligned, nothing decorative that isn't load-bearing.

Commit to:

- **Editorial typography.** Bebas Neue display set large and tight, doing the work that decoration usually does. Type is the design.
- **Hairlines over boxes.** A 1px rule at low opacity separates better than a border and a fill.
- **Deep, calm ground.** Navy `#101820` base with *tinted elevation* — surfaces get lighter by adding a steel-tinted overlay, not by adding grey. This is the difference between "dark mode" and "designed dark".

- **One accent, used sparingly.** Steel `#7BA7BC` earns attention because it is rare. If everything is steel, nothing is.
- **Generous, rhythmic space.** All spacing from one scale. Optical alignment over mathematical where they disagree.
- **Motion as feedback, never as ornament.** Every animation answers "what just happened" or "where did this come from".

Reject:

- Frosted glass on every surface. It is already load-bearing in the mobile sheet — keep it there, and nowhere else.
- Glow shadows on buttons.
- Uniform border-radius everywhere. Radius should encode hierarchy: sharp for structure, rounded for interactive.
- Grey-on-grey admin chrome that looks like a dashboard template.
- Emoji as iconography.

2.2 Principles that settle arguments

1. **The token is the source of truth.** If a value appears twice, it is a token. If it is a token, no component may hard-code it. (Already enforced on web by `admin-ui-kit.test.js`.)
2. **Hierarchy before decoration.** Fix what the eye reaches first, second, third. Only then consider surface treatment.
3. **The empty state is a design, not a fallback.** Every list, panel and workspace gets a composed empty state that tells the user what would fill it and offers the action that does.
4. **Loading has a shape.** Skeletons mirror the real content's geometry. No generic grey bars.
5. **Keyboard-first on desktop, thumb-first on mobile.** Neither is an afterthought of the other.
6. **Motion is opt-out at the token layer.** `prefers-reduced-motion` sets durations to `0ms` in one place, rather than being handled per component. (`public-accessibility-contract.test.js` already tests for reduced-motion awareness.)
7. **Density is a user setting, not a guess.** Byron on an iPad at the desk and Kirra on a phone at the door want different row heights.

3. ARCHITECTURE: ONE TOKEN PIPELINE, THREE CONSUMERS

This is the part that makes the work technically impressive rather than merely pretty, and it must be built **first**.

3.1 The shape

```
design/tokens.json          ← single source of truth, hand-edited
|
├─ scripts/build-tokens.mjs ← deterministic generator, no deps
|
├─ src/styles/tokens.css    (CSS custom properties, 3 tiers)
├─ tailwind.config.tokens.cjs (Tailwind theme reads the vars)
└─ ios/.../Generated/XertTokens.swift (Swift constants + Color extensions)
```

`design/tokens.json` holds three tiers, and the tiering is the whole point:

- **Primitive** — raw values with no meaning. `navy-900: #101820`, `steel-400: #7BA7BC`, `space-4: 1rem`, `dur-quick: 140ms`.
- **Semantic** — meaning, referencing primitives. `surface.base`, `surface.raised`, `surface.overlay`, `text.primary`, `text.muted`, `border.hairline`, `accent.default`, `accent.muted`, `state.danger`, `focus.ring`.
- **Component** — the few places a component legitimately needs its own knob. `nav.height.compact`, `nav.height.expanded`, `card.radius`, `dock.blur`.

Components consume **semantic** tokens only. A component referencing a primitive is a bug, and section 3.4 makes it a failing test.

3.2 Why this is worth the effort

- The palette stops being remembered and starts being imported.
- Web and iOS cannot drift, because drift becomes a red test.
- Theming (a light mode, a high-contrast mode, a seasonal accent) becomes a data change.
- The generated Swift file gives iOS the same semantic vocabulary as the web, which is what makes Workstream D's rewrite feel like one product rather than two.

3.3 Elevation model

Define surfaces as **tinted overlays**, not discrete greys:

```
surface.base      = navy-900
surface.raised    = navy-900 + steel @ 3.5%
surface.overlay   = navy-900 + steel @ 7%
surface.sunken    = navy-900 darkened 4%
border.hairline   = steel @ 12%
border.strong     = steel @ 24%
```

This produces a dark UI where depth reads as *air and light* rather than as a stack of grey rectangles, and it is the single change that will most improve the Command Centre's feel on both platforms.

3.4 Proof required

Add `test/design-tokens.test.js`:

- `design/tokens.json` parses, and every semantic token resolves to a real primitive.
- Running the generator produces output byte-identical to what is committed — so a token change with a stale build fails CI.
- No file under `src/components/` or `src/pages/` contains a raw hex or `rgba(` literal (extends the existing `admin-ui-kit.test.js` rule to the whole app).
- The generated Swift palette and the generated CSS palette contain the same semantic names.

4. WORKSTREAM A – PUBLIC NAVIGATION (WEB)

Files: `src/components/public/PublicNav.jsx` (220 lines), new `src/components/public/nav/` for extracted parts.

A1. Desktop bar

- Taller resting bar (72–80px) that **compresses to ~56px on scroll**, animating height, logo scale and background together. Drive it with a single scroll-progress value and CSS variables so it is one paint, not five transitions.
- **Animated active indicator** that slides between items. Implement with CSS anchor positioning where supported and a FLIP/ `ResizeObserver` fallback; it must move, not cross-fade.
- **Three tiers of weight, visibly distinct:** navigation links (quiet), account (a bordered control), Book Now (the only filled element in the bar). Separate account and CTA from the link group with a hairline divider, not just a gap.
- Replace the glow pill with a flat accent surface and a *tight* shadow that only appears on hover/active.
- Add a **scroll-progress hairline** along the bottom edge of the bar — one line, no chrome, quietly premium.
- The transparent-over-hero state needs a text-shadow or a subtle top scrim so links stay legible over bright imagery.

A2. Mobile sheet — reclaim the dead half

Keep the current structure (it is well built: focus trap, scroll lock, escape handling, safe areas, viewport-crossing cleanup — **preserve all of it**, and note that

`public-accessibility-contract.test.js` tests the trigger's name, expanded state, `aria-controls` and 44px target).

Changes:

- **Delete the chevrons.** They promise a submenu that does not exist.
- Fill the empty lower half with what someone opening this menu actually wants:
- **Next class** — time, coach, spots left, with a direct book/request action.
- **Account strip** — signed in: name, next booking, link to account. Signed out: Log In / Join.
- **Call the gym** — a real `tel:` link. It is a gym; people phone.
- Opening hours or today's status.
- Stagger the entrance on a **spring**, not a linear delay ramp, and gate it behind the reduced-motion token.
- Add **swipe-down-to-dismiss** with rubber-banding, tracking the finger rather than snapping.
- Give the close control the same 44px treatment as the opener.

A3. Cross-cutting

- Route changes get **View Transitions** (`document.startViewTransition`) with a no-op fallback and reduced-motion respect.
- A `prefers-reduced-transparency` fallback for the glass panel.

A4. Proof

Playwright screenshots at 390×844, 768×1024, 1440×900 and 1920×1080 for: resting nav, scrolled nav, open sheet (signed out), open sheet (signed in), and the sheet at 200% text zoom. Extend `public-accessibility-contract.test.js` for the new controls. Contrast-check every new pairing programmatically.

5. WORKSTREAM B — COMMAND CENTRE SHELL (WEB)

Files: `src/components/admin/AdminLayout.jsx`, `src/components/admin/CommandPalette.jsx` (131 lines), `src/lib/adminWorkspaces.js`, `src/pages/AdminCommandCentre.jsx`.

B1. Layout

- **Resizable sidebar** with the width persisted per device, a keyboard-accessible drag handle, and a collapsed icon-rail mode with tooltips.
- **Breadcrumbs** — with 24 sections, "where am I" needs answering above the fold.
- Replace the horizontally scrolling tab rail with a proper **roving-tabindex tablist** with an animated indicator and edge fades that only render when there is actually overflow.
- **Density toggle** (comfortable / compact), persisted, applied via a data attribute that the token layer reads.

B2. The command palette becomes a command system

Today it navigates. It should *act*.

- Fuzzy scoring with match highlighting (subsequence + word-boundary bonus; no dependency needed).
- Recents and frequency weighting, persisted.
- **Actions, not just destinations:** "Confirm booking...", "Text tomorrow's 6:15am", "Add attendee", "Mark attendance", "Publish form" — each with its own arguments resolved inside the palette.
- Nested argument prompts (pick a class → pick a member) without leaving the palette.
- `⌘K` / `Ctrl+K` plus `g`-prefixed sequences (`g c` → calendar) with a discoverable shortcut sheet on `?`.

B3. State and feedback

- **Optimistic mutations with rollback and undo toasts** — the Command Centre is used standing at a door; a confirmation that waits for a round trip is a confirmation that gets pressed twice.
- **Per-region Suspense boundaries** so one slow query does not blank a workspace.
- **Content-shaped skeletons** generated from the same layout primitives as the real rows.
- A real **error boundary** per workspace with a retry that does not lose filter state.
- Preserve the existing unsaved-changes guard in `AdminCommandCentre.jsx` — it is correct and easy to break.

B4. Proof

`admin-navigation.test.js`, `admin-navigation-ia.test.js`, `admin-mobile-navigation.test.js` and `admin-ui-kit.test.js` must stay green. Add palette scoring unit tests and keyboard-only navigation tests through Playwright.

6. WORKSTREAM C — COMMAND CENTRE PANELS (WEB)

24 sections, ~12,900 lines under `src/components/admin/`. Do **not** rewrite them one at a time from scratch. Build the vocabulary first, then convert.

C1. Extend the kit (`src/components/admin/ui.jsx`)

Add, with the same test discipline that guards the existing kit:

`AdminDataTable` (sortable, selectable, virtualised past ~200 rows, responsive card fallback on phones) · `AdminFilterBar` (URL-synced filter state) · `AdminStatCard` · `AdminEmptyState` · `AdminSkeleton` · `AdminBadge` (one status-colour map, derived from tokens — today `LeadTable.jsx` owns a private one) · `AdminDrawer` · `AdminFormField` · `AdminSegmented` · `AdminTimeline`.

C2. Container queries

Admin panels sit in a column whose width depends on the sidebar, not the viewport. Style them with `@container`, not media queries. This is the correct tool and almost nobody reaches for it.

C3. Conversion order

Ordered by daily use, from what the team has actually been reporting:

1. `ClassCalendarAdmin` + the class roster (1,555 lines — the screen Kirra lives in)
2. `LeadTable` (336)
3. `FormsSurveysManager` + `FormResponseRecord` (293 + 217)
4. `MembersManager` (1,243)

5. `OrdersManager` (431)
6. `AdminToday` / overview
7. Everything else

Convert one panel per commit, screenshot before and after, keep the suite green.

7. WORKSTREAM D – IOS COMMAND CENTRE (THE MAIN EVENT)

File: `AdminCommandCentreView.swift`, 18,009 lines. This is the work that answers the actual complaint.

D1. Grow the design system first

Expand `AdminDesignSystem.swift` (117 lines) into a real system consuming generated `XertTokens.swift`:

- **Type ramp** — display / title / heading / body / caption / mono, each Dynamic-Type aware. (`native-safe-area-layout.test.js` already requires heroes to expand rather than clip at accessibility sizes — honour it.)
- **Surface primitives** — `XertCard`, `XertSurface`, `XertHairline`, using the tinted elevation model from §3.3.
- **Controls** — `XertButton` (primary/ghost/danger/quiet), `XertField`, `XertSegmented`, `XertToggleRow`, `XertMenuField`, `XertBadge`, `XertStat`.
- **States** — `XertEmptyState`, `XertSkeleton`, `XertInlineError`.
- **Motion** — one spring vocabulary, plus systematised `XertHaptics` pairings.

D2. Structural rewrite

The core move, repeated across the file:

```
List { Section("...") { ... } } .listRowBackground(...)
↓
ScrollView { LazyVStack(spacing: XertSpace.md) { XertCard { ... } } }
```

Targets: **130 Sections** → **0**, **35 Lists** → **0** (except genuinely swipe-actionable rosters), **13 Forms** → **0**, **174 .listRowBackground** → **0**, **78 .bordered buttons** → `XertButton`, **56 stock Picker** → `XertSegmented` / `XertMenuField`, **49 TextField** → `XertField`.

D3. Custom navigation chrome

- Replace 51 `.navigationTitle` calls with a **branded collapsing header**: large Bebas title that shrinks into an inline bar on scroll, driven by scroll offset with `matchedGeometryEffect`.
- Workspace switching gets a real transition, not a push-pop.
- Keep the existing `NavigationSplitView` on iPad — restyle the sidebar with the new primitives rather than replacing the structure.

D4. Sequencing inside the file

18k lines cannot land in one commit. Split by workspace, in this order — each one compiles, runs and ships on its own:

1. Design system expansion (no behaviour change)
2. Dashboard / Today
3. Class roster + attendance (the screens in daily use)
4. Leads
5. Members
6. Forms
7. Settings, health, audit
8. Navigation chrome last, once every surface can carry it

Extract each workspace into its own file as you convert it. A file this size is itself a defect; finishing at 18k lines in one file would be a missed opportunity.

D5. Proof

`native-owner-design-system.test.js` (6 tests) is your friend here — it already asserts every owner screen sits on the shared backdrop, cards are padded from the scale, and primitives delegate to the system. **Extend it** with assertions that the Command Centre contains no `Section()`, no `Form {}`, and no `.ListRowBackground`. That turns "don't regress to stock iOS" into a test rather than a hope.

❗ **The environment that wrote this plan has no Swift compiler.** Every iOS change must be built in Xcode and run on a device or simulator before it is called done. Screenshot each converted workspace at default and at accessibility text sizes.

8. WORKSTREAM E — IOS MEMBER SURFACES

Lower priority: `RootView.swift` already hides the system tab bar and renders a custom dock and rail, so the foundation is right. Once D is done, sweep `HomeView`, `BookingView`, `EventsView`, `ExploreView` and `AccountView` for the same stock-`List` pattern and apply the new primitives so the member app and the owner app read as one product.

Note for whoever picks this up: `AccountView.swift` still renders a **class-credit wallet** (`creditSummary`, `creditBatchWallet`, "Session credits & packs"). Credits were retired — they are now off behind `admin_settings.class_credits_enabled`. The web already hides the wallet unless the member holds legacy credits; iOS should do the same.

9. SEQUENCE

STAGE	WORK	DEPENDS ON	SHIPS
0	§3 token pipeline + <code>design-tokens.test.js</code>	—	Invisible; everything else rests on it
1	A — public navbar	0	Visible win, small surface, high confidence
2	B — web admin shell	0	The frame the panels live in
3	D1 — iOS design system	0	No behaviour change, unblocks D2
4	C1-C2 — web kit + container queries	2	Vocabulary for the panels
5	D2-D4 — iOS workspaces, in order	3	Ships per workspace
6	C3 — web panels, in order	4	Ships per panel
7	E — iOS member sweep	5	Consistency pass

Stages 4/5 and 6/5 can run in parallel across two people — one web, one iOS — once stage 3 lands, because the token layer keeps them honest.

Do not start stage 1 before stage 0. Everything downstream depends on there being one place to change a colour.

10. WORKING NOTES FOR WHOEVER EXECUTES THIS

10.1 Verification recipes

Dev server needs Supabase env vars or every data-backed screen renders "unavailable":

```
VITE_SUPABASE_URL="https://<ref>.supabase.co" VITE_SUPABASE_ANON_KEY="<anon key>" \  
npm run dev -- --port 5185 --host 127.0.0.1
```

Screenshots. The headless browser has **no direct internet**, so Supabase calls must be relayed through Node with `page.route`:

```
import { chromium } from 'playwright-core';  
const b = await chromium.launch({  
  executablePath: '/opt/pw-browsers/chromium-1194/chrome-linux/chrome',  
  args: ['--no-sandbox'],  
});  
const p = await (await b.newContext({ viewport: { width: 390, height: 844 }, isMobile: true }));  
await p.route(u => u.hostname.endsWith('.supabase.co'), async route => {  
  const req = route.request(); const h = { ...req.headers() };  
  delete h.host; delete h['content-length'];  
  const res = await fetch(req.url(), {  
    method: req.method(), headers: h,  
    body: ['GET', 'HEAD'].includes(req.method()) ? undefined : req.postData(),  
  });  
  const body = Buffer.from(await res.arrayBuffer()); const rh = {};  
  res.headers.forEach((v, k) => {  
    if (!['content-encoding', 'transfer-encoding', 'content-length'].includes(k)) rh[k] = v;  
  });  
  await route.fulfill({ status: res.status, headers: rh, body });  
});
```

Screenshot **every** change at 390×844, 768×1024 and 1440×900, plus one pass at 200% text zoom.

Gates — all three, every commit:

```
npm test          # 1,110 structural tests  
npm run lint      # eslint --quiet, zero warnings  
npm run build     # also verifies the PWA precache manifest
```

10.2 Guardrails that will bite you

- **The test suite is structural.** Many of the 1,110 tests read source files and assert on regexes and orderings — renaming a component or reordering guards fails tests that look unrelated. Read the failing assertion before assuming you broke behaviour; often the test encodes a real invariant worth preserving.
- `test/admin-ui-kit.test.js` **forbids** hand-rolled primary buttons, input classes, page titles, raw hex/rgba, and off-scale opacity modifiers. Extend the kit; never work around it.
- `test/native-owner-design-system.test.js` **forbids** owner screens on flat navy, raw padding numbers, and primitives that re-implement the system.
- `test/public-accessibility-contract.test.js` pins the mobile nav trigger's accessible name, expanded state, `aria-controls` and 44px minimum, plus reduced-motion behaviour.
- **Vercel Hobby caps serverless functions at 12.** Do not add files to `api/`; multiplex into an existing handler.
- **Never commit secrets.** Browser-exposed config must be `VITE_`-prefixed and injected at build time.

10.3 Git conventions

Develop on the assigned feature branch. Push with `git push origin HEAD:main`, then `git push -f origin HEAD:<branch>`. Commit as:

```
git -c user.name="Claude" -c user.email="noreply@anthropic.com" commit
```

with the `Co-Authored-By` and `Claude-Session` trailers used elsewhere in the history. Do not open a PR unless asked. Do not put model identifiers in anything pushed to the repo.

10.4 Definition of done, per stage

1. Screenshots at all three widths, before and after, attached to the commit or the reply.
 2. `npm test`, `npm run lint`, `npm run build` green — and for iOS, an Xcode build that runs.
 3. New behaviour covered by a test that would fail without it.
 4. No new raw colour, spacing or duration literal anywhere.
 5. Keyboard-only pass on web; VoiceOver pass on iOS.
 6. Reduced-motion and 200% text-zoom both verified.
-

A. APPENDIX — FILE MAP

AREA	FILE	LINES
Public nav	<code>src/components/public/PublicNav.jsx</code>	220
Admin shell	<code>src/components/admin/AdminLayout.jsx</code>	375
Admin router	<code>src/pages/AdminCommandCentre.jsx</code>	179
Admin nav model	<code>src/lib/adminNavigation.js</code>	51
Web UI kit	<code>src/components/admin/ui.jsx</code>	64
Command palette	<code>src/components/admin/CommandPalette.jsx</code>	131
Admin panels	<code>src/components/admin/*.jsx</code>	~12,900
iOS Command Centre	<code>.../Views/AdminCommandCentreView.swift</code>	18,009
iOS member root	<code>.../Views/RootView.swift</code>	2,289
iOS navigation	<code>.../XertNavigation.swift</code>	1,669
iOS theme	<code>.../Theme.swift</code>	469
iOS design system	<code>.../AdminDesignSystem.swift</code>	117

Largest single opportunity: `AdminCommandCentreView.swift` — 18,009 lines, 130 stock Sections, 174 `.listRowBackground` calls, and a design system sitting right next to it that it uses for exactly one thing.